



Missão Prática | Nível 4 | Mundo 3

Claudio Daniel Pereira Carvalho Leite | 202401506958

Polo Cremação - Belém - PA

BackEnd sem banco não tem – 2025.1 – 3º Semestre

Objetivo da Prática

A atividade prática teve como objetivo a construção de uma aplicação full stack multi-camada, totalmente containerizada, com separação clara de responsabilidades entre as camadas. Trata-se de um sistema modular, independente de plataforma ou sistema operacional, viabilizado pelo uso do Docker para a orquestração e o deploy dos serviços.

Na **camada de dados**, utilizou-se o SGBD Microsoft SQL Server. Nas camadas de persistência (ou de **acesso a dados**) e de aplicação (ou de **lógica de negócio**), foi utilizado o framework Spring Boot, responsável pelo gerenciamento das dependências e bibliotecas. O acesso ao banco de dados foi realizado por meio do driver JDBC, utilizando o mapeamento objeto-relacional (ORM) conforme o padrão JPA, com a especificação Jakarta EE utilizando a biblioteca Hibernate.

A aplicação Spring Boot é executada em um servidor Tomcat embutido, o qual atua como servidor de aplicações leve, sendo responsável pelo tratamento de requisições HTTP, gerenciamento de servlets, sessões e integração com o backend. Ao utilizar o Tomcat embutido, elimina-se a necessidade de um servidor de aplicações tradicional como JBoss ou GlassFish, visto que o próprio Spring Boot já incorpora as funcionalidades necessárias em um único artefato executável (JAR). Essa abordagem torna a aplicação mais leve, portátil e adequada para ambientes containerizados.

Na camada cliente (ou camada de apresentação), empregou-se a biblioteca React, com ferramentas como Node.js, pnpm e Vite para o empacotamento (bundle) e a construção (build) da interface do usuário. Ainda na camada de apresentação, emprega-se o NGINX como servidor web e proxy reverso, responsável por servir os arquivos estáticos gerados pelo React (build) e encaminhar as requisições de API para o backend em Spring Boot. Essa separação de responsabilidades entre NGINX (frontal) e Tomcat (backend) melhora a escalabilidade, a segurança e o desempenho da aplicação.

1º Procedimento | Camadas de persistência e lógica de negócio

Codigos:

Dependências :

1. Pom.xml:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/pom.xml

Configurações de conexão da camada de persistência com a camada de dados:

2. Application.yml:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/src/main/resources/application.yml

Pacote model.entity :

3. Produto.java:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/src/main/java/com/loja/model/entity/Produto.java

Pacote model.repository :

4. ProdutoRepository.java:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/src/main/java/com/loja/model/repository/ProdutoRepository.java

Pacote loja.service:

5. ProdutoService.java:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/src/main/java/com/loja/service/ProdutoService.java

Pacote loja.controller:

6. ProdutoController.java:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/src/main/java/com/loja/controller/ProdutoController.java
7. RootController.java:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/src/main/java/com/loja/controller/RootController.java

Configurações de conexão da camada de aplicação com a camada cliente:

8. WebConfig.java
https://github.com/claudio-py/loja_java_crud/blob/v2/loja-app/src/main/java/com/loja/config/WebConfig.java

2º Procedimento | Camada de apresentação

Codigos:

Servidor da camada :

9. ProdutoRepository.ts:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja_react/nginx.conf

Views :

10. NovoProduto.tsx :
https://github.com/claudio-py/loja_java_crud/blob/v2/loja_react/src/view/NovoProduto.tsx

11. ProdutoList.tsx :
https://github.com/claudio-py/loja_java_crud/blob/v2/loja_react/src/view/ProdutoList.tsx

React Hooks :

12. NovoProduto.tsx :
https://github.com/claudio-py/loja_java_crud/blob/v2/loja_react/src/hooks/useProdutos.ts

Model :

13. Produto.ts:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja_react/src/model/Produto.ts

Repositoryy :

14. ProdutoRepository.ts:
https://github.com/claudio-py/loja_java_crud/blob/v2/loja_react/src/repository/ProdutoRepository.ts

3º Resultados finais:

- Visualização dos endpoints com swagger em <http://localhost:8081>

The image is a screenshot of the Swagger UI interface. It displays three API controllers and their endpoints:

- produto-controller**
 - GET /api/produtos
 - PUT /api/produtos
 - POST /api/produtos
 - DELETE /api/produtos
 - GET /api/produtos/getby
- pessoa-juridica-controller**
 - GET /api/pessoas/juridica
 - POST /api/pessoas/juridica
- pessoa-fisica-controller**
 - GET /api/pessoas/fisica
 - POST /api/pessoas/fisica

Below the controllers, there is a **Schemas** section containing three expandable schema definitions:

- Produto** > Expand all **object**
- PessoaJuridica** > Expand all **object**
- PessoaFisica** > Expand all **object**

- Listar/excluir/alterar/buscar produtos: <http://localhost:3000/Produtos>

|Listar Produtos|

|Novo|

Lista de Produtos

Buscar por ID

Buscar

Id	Nome	Estoque	Valor de Venda	Valor de Custo	Ações
1	Teclado	50	70	50	<button>Alterar</button> <button>Excluir</button>
2	Mouse	80	45	30	<button>Alterar</button> <button>Excluir</button>
3	Monitor	30	550	400	<button>Alterar</button> <button>Excluir</button>
4	Notebook	15	2500	2000	<button>Alterar</button> <button>Excluir</button>
5	Impressora	20	800	600	<button>Alterar</button> <button>Excluir</button>
6	Webcam	40	160	120	<button>Alterar</button> <button>Excluir</button>
7	Headset	60	210	150	<button>Alterar</button> <button>Excluir</button>
8	HD Externo	25	400	300	<button>Alterar</button> <button>Excluir</button>
9	Pen Drive	100	60	40	<button>Alterar</button> <button>Excluir</button>
10	Cabo HDMI	70	35	25	<button>Alterar</button> <button>Excluir</button>

- Cadastrar novo produto

|Listar Produtos|

|Novo|

Incluir Produto

Nome:

Estoque:

Preço de Venda:

Valor de Custo:

Incluir